

CMS

INTERNSHIP PROJECT REPORT

Blog Content Management System

(Blog CMS) — Full-Stack Web Application

Aditya Raj Saxena

Roll No. 2410030189 | B.Tech CSE | Semester V | Batch 2024–2028

School of Computer Science & Engineering, IILM University, Greater Noida, U.P.

In collaboration with AICTE • UniConverge Technologies • upSkill Campus

Internship Completion Date: 21 August 2026

What We'll Cover Today

1

Introduction & Overview

2

Internship & Organization

3

Problem Statement

4

Project Objectives & Scope

5

Technology Stack

6

System & Database Architecture

7

API Design

8

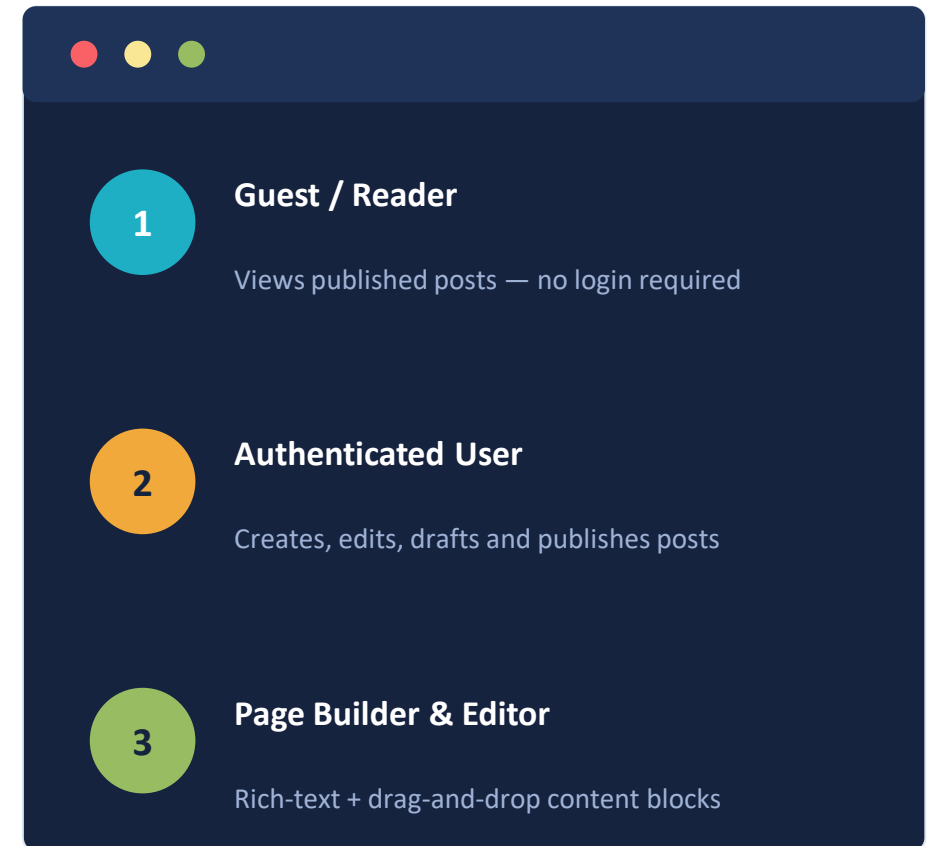
Development Process & Progress

9

Security, Testing & Outcomes

What is the Blog CMS?

- A Content Management System (CMS) lets users create, edit and publish content without touching application code.
- Blog CMS provides a public area for readers and a secure dashboard for authors.
- Built as a full-stack app: React frontend, Node.js/Express backend, MongoDB database.
- Includes a rich-text editor and a drag-and-drop page builder for flexible content.
- Authentication (JWT) and authorization keep management actions secure.



Organization & Engagement Profile

1

Provider

upSkill Campus, with industry partner UniConverge Technologies Pvt. Ltd.

2

Role

Full Stack Development Intern

3

Collaboration

All India Council for Technical Education (AICTE)

4

Mode & Hours

Remote engagement, 10:00 AM – 6:00 PM

INTERNSHIP TIMELINE

4 Weeks | 25 July – 21 August 2026



Why Build a Unified Blog CMS?

Traditional blog workflows scatter content creation, layout, storage and publishing across separate tools or manually maintained code — making them hard to maintain and secure.

Goal: a single browser-based system where authenticated users manage content, while readers get a clean public interface — with structured data and controlled access throughout.

KEY REQUIREMENTS

- ✓ Secure registration & login for authors
- ✓ Token-based authentication & authorization
- ✓ Full CRUD for blog posts via REST APIs
- ✓ Draft and published post states
- ✓ Rich-text formatting for structured content
- ✓ Drag-and-drop page builder with reusable blocks

What the Internship Set Out to Achieve

1

Client–Server Architecture

Apply React, Node.js, Express.js and MongoDB together

2

Modular Frontend

Reusable navigation, dashboard, editor & post components

3

Drag-and-Drop Builder

Heading, text, image, quote & divider blocks via React-DnD

4

Secure REST APIs

Full CRUD operations for blog posts

5

JWT Authentication

Registration, login & bcrypt password hashing

6

Testing & Version Control

Postman API testing plus structured Git/GitHub history

Areas Covered in the Build

1

User Management

Registration, login, JWT auth, bcrypt hashing

2

Blog Management

Create, read, update, delete; draft & published states

3

Content Editing

Rich-text editing and formatted storage

4

Page Builder

Drag-and-drop heading, text, image, quote, divider blocks

5

Backend

Express server, REST routes, middleware, validation

6

Database

MongoDB with Mongoose models for users & posts

7

Frontend

React, React Router, reusable dashboard & public pages

8

Testing

Postman API tests, responsive & end-to-end checks

Tools Powering the Blog CMS

FRONTEND

- React.js
- React Router
- React-DnD
- React Quill

BACKEND

- Node.js
- Express.js
- JWT
- bcryptjs

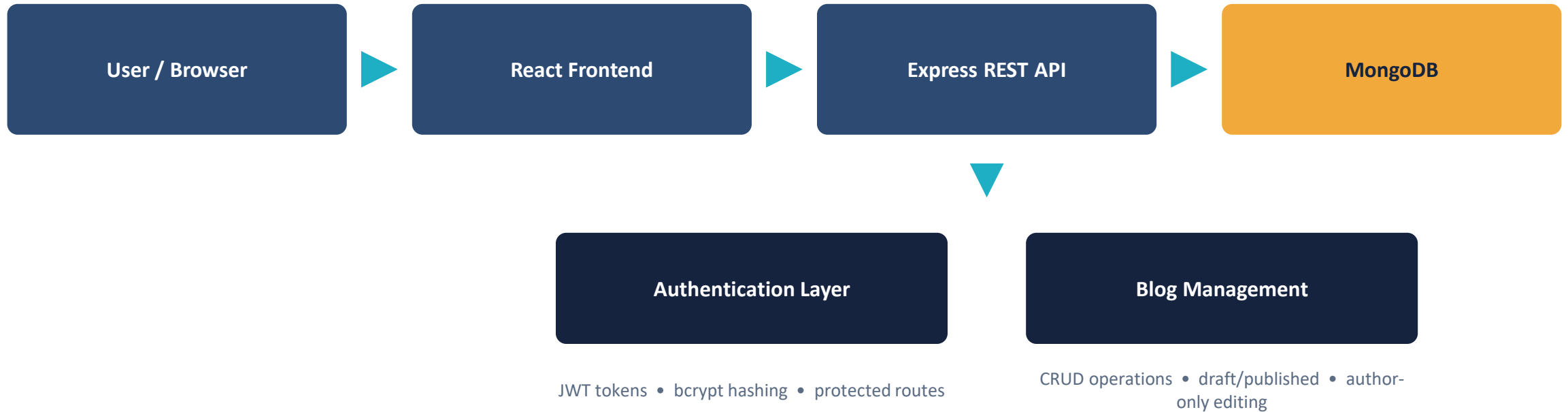
DATABASE

- MongoDB
- Mongoose

TOOLING

- Postman
- Git & GitHub
- VS Code

Layered Client-Server Design



The frontend talks to the backend only through REST APIs. The backend applies validation, authentication, authorization and business logic before touching the database.

Entity-Relationship Overview



Each blog post stores a reference to its author's User ID — the server uses this link to restrict update and delete actions to the post's owner.

Data Model Validation Rules

User Model

Field	Purpose / Validation
name	Required; trimmed; 2–60 characters
email	Required, unique, lowercase & trimmed
password	Required; min 6 chars; stored as bcrypt hash
createdDate	Auto-assigned on record creation

BlogPost Model

Field	Purpose / Validation
title	Required, trimmed; 3–160 characters
content	Required; stores rich-text content
author	Required ObjectId reference to User
publishStatus	Enum: draft published (default draft)
updatedAt	Updated via pre-save hook on each save

Blog CMS REST Endpoint Reference

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create a new user account
POST	/api/auth/login	Public	Verify credentials, return JWT
GET	/api/posts	Public	List published posts (newest first)
GET	/api/posts/:id	Public / Owner	Return a single post
GET	/api/posts/mine	Authenticated	Return the current user's posts
POST	/api/posts	Authenticated	Create a draft or published post
PUT	/api/posts/:id	Author only	Update title, content, status
DELETE	/api/posts/:id	Author only	Delete a blog post

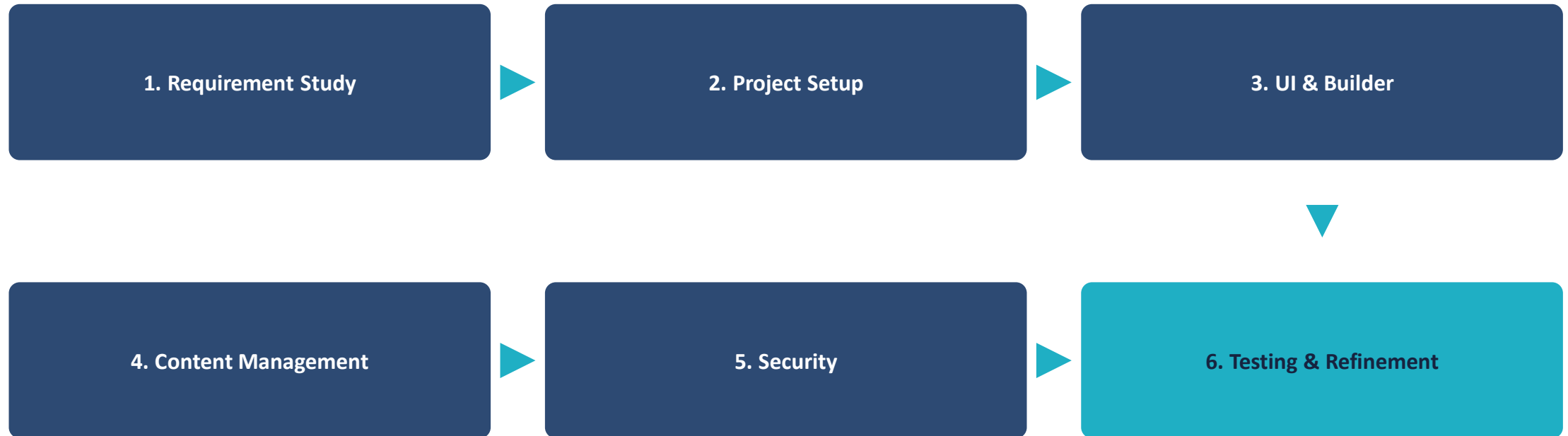
Protected requests carry an Authorization: Bearer <token> header. A middleware layer verifies the JWT signature before any protected controller runs.

Blog Post CRUD Workflow



All four operations are protected by authentication, server-side author authorization, and Mongoose schema validation — a user can only edit or delete their own posts.

Progressive, Feature-by-Feature Methodology



Frontend, backend, database, authentication, content features and testing were implemented and verified incrementally across four weeks.

Four-Week Development Summary

Week 1

Planning & Setup

- Requirements & tech selection
- Dev environment configured
- Git/GitHub initialized
- React-DnD explored

Week 2

Core Architecture

- React dashboard & components
- Express server & routes
- MongoDB connected
- Blog-post schema designed

Week 3

Content Management

- CRUD operations implemented
- Rich-text editor integrated
- Forms connected to APIs
- Postman API testing

Week 4

Security & Finalization

- JWT login & registration
- bcrypt password hashing
- Responsive UI fixes
- End-to-end testing

Authentication & Access Controls



Password Hashing

bcryptjs with a work factor of 12



JWT Authentication

Signed, time-limited session (2-hour expiry)



Protected Routes

Express middleware checks Bearer token



Author Authorization

Post author ID compared to req.user ID



Input Validation

Required fields, string limits, status enum



Environment Secrets

.env excluded via .gitignore

Representative Test Matrix

Test Area	Representative Check	Expected Result
Registration	Valid name, email & password	Account created, token returned
Duplicate Email	Register with existing email	Rejected with conflict message
Login	Valid vs. invalid credentials	Accept / reject correctly
Protected Route	Request without token	Rejected as unauthorized
Author Restriction	User B edits User A's post	Rejected as forbidden
CRUD Workflow	Create, edit, retrieve, delete	State changes persist correctly
Responsive UI	Desktop vs. smaller viewport	Layout remains usable

Testing combined Postman for API verification, browser DevTools for debugging, and responsive/device checks for the interface.

Issues Faced During Development



API Integration Errors

Issue: Request/validation errors during frontend–backend integration

Fix: Compared payloads using Postman & browser DevTools



CRUD Routing Issues

Issue: Some update/delete calls failed due to route & middleware order

Fix: Reviewed Express routing and MongoDB queries



Auth State Sync

Issue: React components didn't reflect session changes after token expiry

Fix: Centralized authentication state via React Context



Responsive Layout Bugs

Issue: Drag-and-drop builder behaved inconsistently on small screens

Fix: Used responsive mode & device testing to fix breakpoints

What the Internship Delivered

- ✓ Functional React frontend with public & dashboard views
- ✓ Structured Express.js backend with auth & blog routes
- ✓ MongoDB integration via Mongoose models & validation
- ✓ Working blog CRUD with draft/published states
- ✓ Rich-text editor + drag-and-drop page builder
- ✓ JWT authentication with bcrypt password hashing
- ✓ Author-level authorization on update/delete
- ✓ Responsive layouts tested across screen sizes

A reusable, feature-complete foundation for future CMS capabilities: roles, media handling, search, analytics and deployment.

CONCLUSION

A Practical, End-to-End Learning Experience

The Blog CMS internship moved from requirements and setup through implementation, security, debugging, testing and final readiness — connecting classroom concepts to real full-stack development practice.

FUTURE SCOPE

- Role-based administration
- Media uploads & categories
- Search, filtering & comments
- Analytics & production deployment

Thank You

Questions & Discussion Welcome

Aditya Raj Saxena | 2410030189

B.Tech CSE, IILM University, Greater Noida